

Logic and Reasoning for Computing

×

Module 1

Propositional Logic

^

Lesson 1.0 Introduction

✔

Course structure and navigation

Reading • 15 min

✔

How to learn effectively on this course

Reading • 15 min

✔

Introduction to the course

Video • 1 min

✔

Course Syllabus

Reading • 10 min

Lesson 1.1 Propositional Logic

●

Introduction to propositional logic

Video • 2 min

●

Building blocks of logic

Video • 6 min

●

Write in logic

Practice Assignment • 20 min

●

Truth table – examples

Video • 8 min

●

Truth table

Practice Assignment • 20 min

Lesson 1.2 Tautology

●

Tautology and consistency (part 1)

Video • 3 min

●

Tautology and consistency (part 2)

Video • 3 min

●

Tautology and consistency – examples

Video • 2 min

●

Tautology or contradiction

Practice Assignment • 25 min

●

Practice Problems: Exercises hints and tips

Reading • 10 min

Lesson 1.3 First-order Logic

●

Equivalences (part 1)

Video • 5 min

●

Equivalences (part 2)

Video • 3 min

●

First-order logic

Video • 8 min

●

Quantifiers

Practice Assignment • 35 min

Lesson 1.4 Summary and assessments

●

Logic

Practice Assignment • 35 min

●

Exercises, hints and tips

Reading • 1h

●

Evaluating Propositional Logic Mastery

Dialogue • 30 min

●

Conclusion

Video • 2 min

●

Check your understanding: End of module 1

Graded Assignment • 20 min

Module 2

Predicate logic

▼

Module 3

Boolean Algebra

▼

Module 4

Proof techniques

▼

Course Syllabus

0:16 / 6:22

2x

Course Description

Logic forms the backbone of computer science, providing the language and methods for precise reasoning, rigorous analysis, and formal proof. Logic and Reasoning for Computing equips learners with essential tools to represent statements, analyse arguments, and verify correctness. Across four modules—Propositional Logic, Predicate Logic, Boolean Algebra, and Proof Techniques—you will build a solid foundation in formal reasoning and connect abstract concepts directly to computing practice. You'll explore truth tables, quantifiers, Boolean operations, and methods of proof, applying them to areas such as programming, digital circuits, and algorithm verification. By the end of this course, you'll not only be able to reason critically and communicate arguments clearly, but also have the confidence to apply logical structures to both theoretical and practical problems in computer science. As the culminating course in the Essential Mathematics for Computer Science specialisation, this course integrates and builds upon the mathematical foundations established in previous courses, providing the formal reasoning framework that underpins algorithm design, programming languages, and computational theory.

Target Audience

This course is designed for computer science students, software developers, and computing professionals who need to develop strong logical reasoning skills. It's ideal for:

- Computer science students seeking to strengthen their foundation in formal logic
- Software developers who want to improve their program verification and debugging skills
- System designers working on complex algorithms and computational processes
- Anyone interested in the formal underpinnings of computing theory and practice
- Professionals preparing for advanced study or research in computer science

Basic mathematical knowledge and problem-solving skills are recommended, with exposure to set theory and functions (as covered in earlier specialization courses) being beneficial but not strictly required.

Content Differentiation

This course stands apart from traditional logic courses by consistently connecting formal logical concepts to practical computing applications. Rather than presenting abstract theory in isolation, each topic is directly tied to its implementation in computing contexts:

- Truth tables are linked to digital circuit design and conditional expressions in programming
- Predicate logic is connected to database queries and formal specifications
- Boolean algebra is applied to circuit simplification and algorithm optimization
- Proof techniques are practiced in the context of algorithm correctness verification

The curriculum emphasizes both theoretical understanding and practical application, using interactive exercises, visual tools, and programming examples to reinforce logical concepts. You'll develop not just the ability to manipulate logical expressions, but also to apply logical reasoning to analyze and solve real computational problems.

Course Goals and Objectives

Upon successful completion of this course, you will be able to:

- Represent natural language statements using formal propositional and predicate logic
- Construct and analyze truth tables to evaluate logical expressions and arguments
- Apply quantifiers to express complex statements about collections of objects
- Use Boolean algebra to simplify logical expressions and circuit designs
- Implement formal proof techniques including direct proof, contradiction, and induction
- Verify the correctness of algorithms and computational procedures
- Translate between formal logical notation and programming constructs
- Apply logical reasoning to analyze and solve computational problems

Course Outline

The course consists of four modules that progressively build your logical reasoning capabilities:

Module 1: Propositional Logic Introduces the fundamental building blocks of logic, including propositions, logical operators, truth tables, and tautologies, establishing the formal language for representing and analyzing statements.

Module 2: Predicate Logic Extends propositional logic with quantifiers to express more complex statements about collections of objects, providing tools for precise specification and reasoning.

Module 3: Boolean Algebra Focuses on the algebraic structure underlying digital systems, connecting symbolic logic to electronic circuits and computational implementations.

Module 4: Proof Techniques Explores formal methods for establishing truth, including direct proof, proof by contradiction, and mathematical induction, essential skills for algorithm verification.

Learning Objectives

Module 1: Propositional Logic

By the end of this module, you will:

- Explain and apply basic concepts of propositional logic
- Construct truth tables and use them to demonstrate logical equivalence
- Translate between natural language statements and symbolic logical expressions
- Identify tautologies, contradictions, and consistent statements
- Apply equivalence laws to simplify logical expressions

Module 2: Predicate Logic

By the end of this module, you will:

- Describe the basic concepts of predicate logic
- Apply universal and existential quantifiers to express complex statements
- Assign truth values to quantified statements
- Apply De Morgan's laws for quantifiers
- Use rules of inference with predicates and quantifiers to construct valid arguments

Module 3: Boolean Algebra

By the end of this module, you will:

- Write and manipulate Boolean expressions
- Apply laws of Boolean algebra to simplify expressions
- Represent Boolean functions using various notations
- Simplify digital logic circuits and expressions
- Convert between truth tables and Boolean expressions
- Implement Boolean functions using logic gates

Module 4: Proof Techniques

By the end of this module, you will:

- Define logical arguments and apply formal proof concepts
- Construct direct proofs for mathematical statements
- Apply proof by contradiction to establish truth
- Use proof by contrapositive as an alternative approach
- Implement mathematical induction to prove statements about sequences
- Recognize and avoid common errors in logical proofs

Go to next item

✔ Completed

Like

Dislike

Report an issue

https://www.coursera.org/learn/logic-and-reasoning-for-computing/supplement/LxFpc/course-syllabus

1/1

Go to next item →